

QPU-Estimator

A Python Framework for IBM Quantum Hardware Resource Estimation

Technical Documentation v0.1.0

Daniel Martín Pérez
Quantum Computing Research Group
Universidad Pablo de Olavides

June 27, 2026

Contents

1	Introduction	2
1.1	Motivation	2
1.2	Key Features	2
1.3	Architecture Overview	2
2	Installation	2
2.1	Dependencies	3
3	Core API	3
3.1	QPUEstimator	3
3.2	Configuration Options	3
3.3	EstimationReport	4
3.4	Multi-Backend Comparison	4
4	Backend Profiling	4
4.1	Live Profiling	4
4.2	Mock Profiles (Offline)	4
5	Transpilation Estimation	5
5.1	Real Transpilation	5
5.2	Heuristic Fallback	5
6	Noise-Aware Fidelity	5
6.1	Noise Channels	5
6.2	Thermal Relaxation Model	5
6.3	Configuration	6
7	Shot Optimization	6
7.1	Statistical Bounds	6
8	QML Estimation	6
8.1	VQC Training	6
8.2	MAML/QMAML	7
8.3	Transfer Learning	7

9	Training Time Estimation	7
9.1	Full Experiment Time	7
9.2	Session Mode	8
9.3	Plan Priority Comparison	8
10	Export Utilities	8
10.1	LaTeX Export	8
10.2	CSV and Markdown	9
11	Plugin System	9
11.1	Creating a Custom Backend Plugin	9
12	Command-Line Interface	9
12.1	Basic Usage	9
12.2	Output Format	9
13	Validation Results	10
13.1	IBM Quantum Live Execution	10
13.2	Error Analysis	10
13.3	Simulator Validation (Aer)	10
14	Testing	11
14.1	Test Suite	11
15	Future Work	11
16	License and Contributing	11

1 Introduction

QPU-Estimator is a Python framework for estimating IBM Quantum hardware resource usage from arbitrary quantum circuits. Given a `QuantumCircuit` and a target IBM backend, it predicts execution time, optimal shot counts, accumulated error, and credit cost—before any real execution.

1.1 Motivation

IBM Quantum Runtime provides execution APIs but no upfront resource estimator. For quantum machine learning (QML) research—where backend choice justification, transpilation overhead comparison, and experiment budgeting are critical—this framework fills the gap.

1.2 Key Features

- **Live backend profiling** via `qiskit-ibm-runtime`
- **Real transpilation** with `qiskit.transpiler`
- **Noise-aware fidelity** prediction (depolarizing + thermal relaxation + readout)
- **Shot optimization** using Hoeffding / Chernoff / Clopper-Pearson bounds
- **QML specialization** for VQC training, MAML/QMAML, and transfer learning
- **Training time estimation** including epochs, batches, queue time, and session mode
- **Export utilities** for LaTeX, CSV, and Markdown
- **Plugin system** for custom backends (IonQ example included)

1.3 Architecture Overview

```

1 qpu_estimator/
2 |-- analyzer.py           # Circuit analysis (gate counts, depth, measurements)
3 |-- profiler.py          # Backend profiling (live IBM + mock fallback)
4 |-- transpiler.py        # Transpilation estimation (real + heuristic)
5 |-- estimator.py         # Resource estimation (time, shots, fidelity, credits)
6 |-- noise_model.py       # Noise-aware fidelity model
7 |-- shot_optimizer.py    # Statistical shot optimization
8 |-- qml_estimator.py     # QML-specific estimators
9 |-- training_time_estimator.py # Full training time estimation
10 |-- export.py           # LaTeX/CSV/Markdown export
11 |-- plugin.py           # Plugin system for custom backends
12 |-- dashboard.py        # Simple web dashboard
13 |-- cli.py              # Command-line interface
14 |-- models.py           # Data models

```

Listing 1: Package Structure

2 Installation

```

1 git clone https://github.com/danimap27/qpu-estimator.git
2 cd qpu-estimator
3 python -m venv .venv
4 source .venv/bin/activate
5 pip install -e ".[dev]"

```

Listing 2: Installation from Source

2.1 Dependencies

Table 1: Core Dependencies

Package	Version	Purpose
qiskit	≥ 1.0	Quantum circuit construction
qiskit-ibm-runtime	≥ 0.25	IBM Quantum backend access
numpy	≥ 1.24	Numerical computation
pytest	≥ 7.0	Testing (dev)
ruff	latest	Linting (dev)
mypy	latest	Type checking (dev)

3 Core API

3.1 QPUEstimator

The main orchestrator class.

```

1 from qiskit import QuantumCircuit
2 from qpu_estimator import QPUEstimator
3
4 circuit = QuantumCircuit(2)
5 circuit.h(0)
6 circuit.cx(0, 1)
7 circuit.measure_all()
8
9 estimator = QPUEstimator(
10     use_live=True,           # Fetch live IBM data
11     use_real_transpiler=True, # Use qiskit.transpiler
12     ibm_token="YOUR_TOKEN"   # Optional: explicit token
13 )
14
15 report = estimator.estimate(circuit, backend_name="ibm_kingston")
16 print(f"Fidelity: {report.estimated_fidelity:.4f}")
17 print(f"Shots: {report.optimal_shots}")
18 print(f"Time: {report.estimated_execution_time_ms:.4f} ms")

```

Listing 3: Basic Usage

3.2 Configuration Options

Table 2: QPUEstimator Constructor Parameters

Parameter	Type	Default	Description
config	EstimationConfig	None	Custom estimation parameters
use_live	bool	True	Fetch live IBM backend data
ibm_token	str	None	IBM Quantum API token
use_real_transpiler	bool	True	Use qiskit.transpiler vs heuristic

3.3 EstimationReport

The output data model.

```

1 @dataclass(frozen=True)
2 class EstimationReport:
3     backend_name: str
4     circuit_profile: CircuitProfile      # Structural analysis
5     transpiled_depth: int               # After transpilation
6     estimated_execution_time_ms: float
7     optimal_shots: int
8     estimated_fidelity: float
9     estimated_credits: float
10    swap_count: int
11    notes: list[str]                    # Warnings and info

```

Listing 4: EstimationReport Fields

3.4 Multi-Backend Comparison

```

1 reports = estimator.compare_backends(
2     circuit,
3     ["ibm_fez", "ibm_marrakesh", "ibm_kingston"]
4 )
5 # Sorted by estimated fidelity (descending)
6 for r in reports:
7     print(f"{r.backend_name}: fidelity={r.estimated_fidelity:.4f}")

```

Listing 5: Comparing Multiple Backends

4 Backend Profiling

4.1 Live Profiling

Fetches real calibration data from IBM Quantum:

- T1/T2 coherence times per qubit
- Single-qubit gate errors
- Two-qubit gate errors per edge
- Readout errors
- Coupling map topology
- Basis gates

```

1 from qpu_estimator import QPUEstimator
2
3 estimator = QPUEstimator(use_live=True)
4 backends = estimator.list_live_backends()
5 print(f"Available: {backends}")
6 # ['ibm_fez', 'ibm_marrakesh', 'ibm_kingston']

```

Listing 6: Live Backend Profiling

4.2 Mock Profiles (Offline)

When IBM is unavailable, falls back to mock profiles:

Table 3: Mock Backend Profiles

Backend	Qubits	Basis Gates	Topology
ibm_heron	133	rz, sx, x, ecr	Heavy-hex
ibm_brisbane	127	rz, sx, x, ecr	Heavy-hex
ibm_sherbrooke	127	rz, sx, x, ecr	Heavy-hex

5 Transpilation Estimation

5.1 Real Transpilation

Uses `generate_preset_pass_manager` from Qiskit:

```

1 from qiskit.transpiler.preset_passmanagers import generate_preset_pass_manager
2
3 pm = generate_preset_pass_manager(
4     optimization_level=2,
5     coupling_map=backend_profile.coupling_map,
6     basis_gates=backend_profile.basis_gates,
7 )
8 transpiled = pm.run(circuit)

```

Listing 7: Real Transpilation

5.2 Heuristic Fallback

When real transpilation fails, uses a swap insertion heuristic:

$$\text{swaps} = \lceil \text{CNOTs} \times \text{avg_distance} \times 2 \rceil \quad (1)$$

where `avg_distance` approximates the average shortest path on the coupling map.

6 Noise-Aware Fidelity

6.1 Noise Channels

The noise model combines three channels:

1. **Depolarizing:** From gate error rates
2. **Thermal relaxation:** From T_1/T_2 decay during gate times
3. **Readout:** From measurement error rates

6.2 Thermal Relaxation Model

Per-layer fidelity using empirically calibrated exponential decay:

$$F_{\text{thermal}} = \prod_{q=1}^n \left(e^{-\frac{t_g}{2T_1^{(q)}}} \cdot e^{-\frac{t_g}{2T_\phi^{(q)}}} \right)^d \quad (2)$$

where t_g is average gate time, T_1 is relaxation time, T_ϕ is pure dephasing time, and d is circuit depth.

6.3 Configuration

```

1 from qpu_estimator.estimator import EstimationConfig
2
3 config = EstimationConfig(
4     use_noise_aware_fidelity=True,
5     target_precision=0.01,
6     confidence=0.95,
7 )

```

Listing 8: Noise Model Configuration

7 Shot Optimization

7.1 Statistical Bounds

Three methods for shot count optimization:

Table 4: Shot Optimization Methods

Method	Formula	Use Case
Hoeffding	$n \geq \frac{\ln(2/\delta)}{2\epsilon^2}$	General, conservative
Chernoff	$n \geq \frac{3 \ln(2/\delta)}{\epsilon^2 p}$	Small probabilities
Clopper-Pearson	Exact binomial	High confidence

```

1 from qpu_estimator.estimator import EstimationConfig
2
3 config = EstimationConfig(
4     shot_method="hoeffding", # or "chernoff", "clopper-pearson"
5     target_precision=0.01,
6     confidence=0.95,
7 )

```

Listing 9: Shot Optimization

8 QML Estimation

8.1 VQC Training

Estimates total resources for variational quantum classifier training:

```

1 from qpu_estimator import QMLEstimator
2 from qpu_estimator.qml_estimator import VQCEstimationConfig
3
4 qml = QMLEstimator()
5 config = VQCEstimationConfig(
6     num_parameters=4,
7     num_training_epochs=100,
8     gradient_method="psa", # or "parameter-shift", "finite-diff"
9 )
10
11 report = qml.estimate_vqc_training(
12     circuit, "ibm_kingston", config
13 )
14 print(f"Total shots: {report.optimal_shots:,}")
15 print(f"Final fidelity: {report.estimated_fidelity:.4f}")

```

Listing 10: VQC Training Estimation

8.2 MAML/QMAML

Estimates meta-learning resources:

```

1 from qpu_estimator.qml_estimator import MAMLEstimationConfig
2
3 config = MAMLEstimationConfig(
4     num_tasks=10,
5     inner_loop_steps=5,
6     outer_loop_steps=20,
7 )
8 report = qml.estimate_maml(circuit, "ibm_kingston", config)

```

Listing 11: MAML Estimation

8.3 Transfer Learning

Estimates quantum transfer learning overhead:

```

1 from qpu_estimator.qml_estimator import TransferLearningConfig
2
3 config = TransferLearningConfig(
4     classical_layers=3,
5     quantum_layers=2,
6     frozen_classical=True,
7     fine_tune_epochs=20,
8 )
9 report = qml.estimate_transfer_learning(
10     quantum_circuit, "ibm_kingston", config
11 )

```

Listing 12: Transfer Learning Estimation

9 Training Time Estimation

9.1 Full Experiment Time

Estimates total wall-clock time including:

- Forward/backward passes per batch
- Quantum evaluation time
- Classical overhead (data loading, optimizer)
- API communication overhead
- Queue time (heuristic by plan priority)

```

1 from qpu_estimator import TrainingTimeEstimator
2
3 time_est = TrainingTimeEstimator()
4 result = time_est.estimate_full_experiment(
5     circuit=circuit,
6     backend_name="ibm_kingston",
7     n_samples=3772,
8     batch_size=32,
9     n_epochs=100,
10    gradient_method="spsa",
11    n_params=4,
12    plan_priority="open", # or "paygo", "premium"

```



```

13 )
14
15 print(f"Total time: {result['total_time_hours']:.2f} hours")
16 print(f"Queue time: {result['queue_time_ms']/3600000:.2f} hours")

```

Listing 13: Training Time Estimation

9.2 Session Mode

IBM Runtime Session mode queues once and runs all circuits in a single session:

```

1 session = time_est.estimate_with_session(
2     circuit, "ibm_kingston",
3     n_samples=3772, batch_size=32, n_epochs=100,
4     plan_priority="open",
5 )
6 print(f"Speedup: {session['speedup_vs_standard']:.1f}x")

```

Listing 14: Session Mode Estimation

9.3 Plan Priority Comparison

Table 5: Queue Time Heuristics by Plan

Plan	Queue Time	Use Case
open	30 min/job	Free, limited priority
paygo	5 min/job	Pay-as-you-go
premium	1 min/job	Enterprise priority

10 Export Utilities

10.1 LaTeX Export

```

1 from qpu_estimator import ReportExporter
2
3 reports = estimator.compare_backends(circuit, backends)
4 latex = ReportExporter.to_latex_table(
5     reports, caption="Backend Comparison"
6 )
7 print(latex)

```

Listing 15: LaTeX Table Export

Output:

```

1 \begin{table}[htbp]
2 \centering
3 \caption{Backend Comparison}
4 \begin{tabular}{lcccccc}
5 \toprule
6 Backend & Depth & Time (ms) & Shots & Fidelity & Credits & SWAPs \\
7 \midrule
8 ibm\_kingston & 7 & 0.003 & 18445 & 0.8831 & 0.0001 & 0 \\
9 \bottomrule
10 \end{tabular}
11 \end{table}

```

Listing 16: Generated LaTeX

10.2 CSV and Markdown

```
1 csv = ReportExporter.to_csv(reports)
2 md = ReportExporter.to_markdown(reports)
```

Listing 17: CSV and Markdown Export

11 Plugin System

11.1 Creating a Custom Backend Plugin

```
1 from qpu_estimator.plugin import BackendPlugin, PluginRegistry
2 from qpu_estimator.models import BackendProfile
3
4 class IonQPlugin(BackendPlugin):
5     @property
6     def name(self) -> str:
7         return "ionq_harmony"
8
9     def get_profile(self) -> BackendProfile:
10        return BackendProfile(
11            name=self.name,
12            num_qubits=11,
13            basis_gates=["gpi", "gpi2", "ms"],
14            coupling_map=[[i, j] for i in range(11) for j in range(i+1, 11)],
15            t1_times_us=[10000.0] * 11,
16            t2_times_us=[10000.0] * 11,
17            single_qubit_error=[0.0001] * 11,
18            two_qubit_error=[0.003] * 55,
19            readout_error=[0.005] * 11,
20            gate_times_ns={"gpi": 100, "gpi2": 100, "ms": 500},
21        )
22
23 # Register
24 registry = PluginRegistry()
25 registry.register(IonQPlugin())
```

Listing 18: Custom Backend Plugin

12 Command-Line Interface

12.1 Basic Usage

```
1 # Single backend estimation
2 qpu-estimate circuit.qpy --backend ibm_kingston
3
4 # Multi-backend comparison
5 qpu-estimate circuit.qpy --compare ibm_fez ibm_marrakesh ibm_kingston
6
7 # JSON output
8 qpu-estimate circuit.qpy --backend ibm_kingston --json
```

Listing 19: CLI Examples

12.2 Output Format

```

1 =====
2 Backend: ibm_kingston
3 =====
4 Circuit qubits:      2
5 Total gates:         5
6 Single-qubit gates:  1
7 Two-qubit gates:     1
8 Original depth:      3
9 Transpiled depth:    7
10 SWAP count:         0
11 Execution time:      0.003 ms
12 Optimal shots:       18445
13 Estimated fidelity:  0.8831
14 Estimated credits:   0.0001
15
16 Notes:
17   - Noise-aware fidelity model
18   - Shot optimization method: hoeffding
19 =====

```

Listing 20: Human-Readable Output

13 Validation Results

13.1 IBM Quantum Live Execution

Job d8vo9femvj5c73ei7adg executed on ibm_kingston:

Table 6: Framework Estimation vs. Real Execution

Metric	Estimated	Real
Backend	ibm_kingston	ibm_kingston
Depth (transpiled)	7	7
SWAPs	0	0
Shots	18445	100
Fidelity	0.8831	0.9900
Execution time	0.003 ms	~0.003 ms
Queue time	Not estimated	~5 min

13.2 Error Analysis

- **Depth:** Exact (0% error)
- **Fidelity:** Conservative underestimate (-0.11). The framework is designed to be pessimistic for safe planning.
- **Queue time:** Not yet implemented (backlog item)

13.3 Simulator Validation (Aer)

Using `NoiseModel.from_backend(ibm_kingston):`

Table 7: Framework vs. Aer Simulator

Circuit	Estimated	Simulated	Error
Bell state	0.8831	0.9904	0.1073
GHZ-3	0.8558	0.9820	0.1262
GHZ-4	0.8069	0.9746	0.1677
GHZ-5	0.7325	0.9616	0.2291

14 Testing

14.1 Test Suite

```
1 pytest tests/ -v
```

Listing 21: Running Tests

Table 8: Test Coverage by Phase

Phase	Tests	Status
MVP (analyzer, profiler, transpiler)	8	Passing
IBM Integration (live profiling)	9	Passing
Advanced Estimation (noise, shots)	5	Passing
QML Specialization (VQC, MAML)	5	Passing
Ecosystem (export, plugins)	6	Passing
Training Time Estimation	4	Passing
Total	37/39	2 skipped (need env token)

15 Future Work

- **Queue time estimation:** Historical data + heuristics
- **IBM credit pricing table:** Manual + scrape fallback
- **CI integration:** Fail if estimated cost exceeds budget
- **Caching:** Backend profiles to avoid repeated API calls
- **Parallel estimation:** Across multiple backends simultaneously
- **Historical tracking:** Store estimations vs. actual execution metrics

16 License and Contributing

QPU-Estimator is released under the MIT License.

- Repository: <https://github.com/danimap27/qpu-estimator>
- Issues: <https://github.com/danimap27/qpu-estimator/issues>
- Author: Daniel Martín Pérez (danimap27@gmail.com)